Practical no 9: Python GUI Application with Local SQLite Database

Question

Design and develop a Python GUI application using Tkinter to connect with a local SQLite database and manage student records. The application should allow the user to add, display, update, and delete student records. Use appropriate GUI widgets and layout management techniques.

Step 1: Create a Python Project

Create a new folder:

`StudentManagement`

Open the folder in your Python IDE.

Create the following Python file:

`student_management.py`

Your project will initially contain:

```
StudentManagement
└── student_management.py
```

Note: The database file will be created automatically when the program is executed.

Step 2: Import Required Modules

Add the following code to `student_management.py`:

```
import tkinter as tk
from tkinter import messagebox
import sqlite3
```

Here:

- `tkinter` → creates the GUI.
- `messagebox` → displays messages.
- `sqlite3` → connects Python to the local SQLite database.

Step 3: Create the Database

Add the following code:

```
connection = sqlite3.connect("students.db")

cursor = connection.cursor()
```

This creates/connects to:

```
students.db
```

The database is stored locally inside the project folder.

Step 4: Create the Student Table

Add:

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    student_id INTEGER PRIMARY KEY,
    name TEXT,
    course TEXT,
    email TEXT
)
""")

connection.commit()
```

The table contains:

Field	Data Type
student_id	INTEGER
name	TEXT
course	TEXT
email	TEXT

Step 5: Create the Main Window

Add:

```
root = tk.Tk()

root.title("Student Management System")

root.geometry("600x550")

root.resizable(False, False)
```

This creates the main GUI window.

Step 6: Add the Heading

```
heading = tk.Label(
    root,
    text="Student Management System",
    font=("Arial", 20, "bold")
)

heading.pack(pady=20)
```

Step 7: Create the Form

Create a frame:

```
form_frame = tk.Frame(root)

form_frame.pack()
```

Student ID

```
tk.Label(
    form_frame,
    text="Student ID:",
    font=("Arial", 12)
).grid(row=0, column=0, padx=10, pady=10)

id_entry = tk.Entry(
    form_frame,
    width=30
)

id_entry.grid(row=0, column=1)
```

Student Name

```
tk.Label(
    form_frame,
    text="Name:",
    font=("Arial", 12)
```

```
).grid(row=1, column=0, padx=10, pady=10)

name_entry = tk.Entry(
    form_frame,
    width=30
)

name_entry.grid(row=1, column=1)
```

Course

```
tk.Label(
    form_frame,
    text="Course:",
    font=("Arial", 12)
).grid(row=2, column=0, padx=10, pady=10)

course_entry = tk.Entry(
    form_frame,
    width=30
)

course_entry.grid(row=2, column=1)
```

Email

```
tk.Label(
    form_frame,
    text="Email:",
    font=("Arial", 12)
).grid(row=3, column=0, padx=10, pady=10)

email_entry = tk.Entry(
    form_frame,
    width=30
)

email_entry.grid(row=3, column=1)
```

Step 8: Create the Add Function

Add this function **before creating the buttons**:

```
def add_student():

    student_id = id_entry.get()
    name = name_entry.get()
    course = course_entry.get()
    email = email_entry.get()

    if student_id == "" or name == "" or course == "" or email == "":
        messagebox.showwarning(
            "Warning",
            "Please fill all fields."
        )
        return
```

```
    try:

        cursor.execute(
            """
            INSERT INTO students
            (student_id, name, course, email)
            VALUES (?, ?, ?, ?)
            """,
            (student_id, name, course, email)
        )

        connection.commit()

        messagebox.showinfo(
            "Success",
            "Student record added successfully."
        )

        clear_fields()

    except sqlite3.IntegrityError:

        messagebox.showerror(
            "Error",
            "Student ID already exists."
        )
```

Step 9: Create the Display Function

```
def display_students():

    text_area.delete("1.0", tk.END)

    cursor.execute(
        "SELECT * FROM students"
    )

    records = cursor.fetchall()

    if len(records) == 0:

        text_area.insert(
            tk.END,
            "No student records found."
        )

    else:

        for record in records:

            text_area.insert(
                tk.END,
                "Student ID: " + str(record[0]) +
                "\nName: " + record[1] +
                "\nCourse: " + record[2] +
                "\nEmail: " + record[3] +
                "\n----------------------------\n"
```

```
            )
```

Step 10: Create the Update Function

```
def update_student():

    student_id = id_entry.get()
    name = name_entry.get()
    course = course_entry.get()
    email = email_entry.get()

    if student_id == "":
        messagebox.showwarning(
            "Warning",
            "Enter Student ID."
        )
        return

    cursor.execute(
        """
        UPDATE students
        SET name=?, course=?, email=?
        WHERE student_id=?
        """,
        (name, course, email, student_id)
    )

    connection.commit()

    if cursor.rowcount > 0:

        messagebox.showinfo(
            "Success",
            "Student record updated successfully."
        )

    else:

        messagebox.showwarning(
            "Warning",
            "Student ID not found."
        )

    clear_fields()
```

Step 11: Create the Delete Function

```
def delete_student():

    student_id = id_entry.get()

    if student_id == "":
        messagebox.showwarning(
            "Warning",
            "Enter Student ID."
```

```
        )
        return

    cursor.execute(
        "DELETE FROM students WHERE student_id=?",
        (student_id,)
    )

    connection.commit()

    if cursor.rowcount > 0:

        messagebox.showinfo(
            "Success",
            "Student record deleted successfully."
        )

    else:

        messagebox.showwarning(
            "Warning",
            "Student ID not found."
        )

    clear_fields()
```

Step 12: Create Clear Function

```
def clear_fields():

    id_entry.delete(0, tk.END)
    name_entry.delete(0, tk.END)
    course_entry.delete(0, tk.END)
    email_entry.delete(0, tk.END)
```

Step 13: Create Buttons

Create a button frame:

```
button_frame = tk.Frame(root)

button_frame.pack(pady=15)
```

Add Button

```
tk.Button(
    button_frame,
    text="Add",
    width=12,
    command=add_student
).grid(row=0, column=0, padx=5)
```

Display Button

```
tk.Button(
    button_frame,
    text="Display",
    width=12,
    command=display_students
).grid(row=0, column=1, padx=5)
```

Update Button

```
tk.Button(
    button_frame,
    text="Update",
    width=12,
    command=update_student
).grid(row=0, column=2, padx=5)
```

Delete Button

```
tk.Button(
    button_frame,
    text="Delete",
    width=12,
    command=delete_student
).grid(row=1, column=0, padx=5, pady=10)
```

Clear Button

```
tk.Button(
    button_frame,
    text="Clear",
    width=12,
    command=clear_fields
).grid(row=1, column=1, padx=5, pady=10)
```

Step 14: Create the Display Area

Add a multi-line Text widget:

```
text_area = tk.Text(
    root,
    width=65,
    height=12
)

text_area.pack(pady=10)
```

Step 15: Run the Application

At the end of the program, add:

```
root.mainloop()
```

Run:

```
student_management.py
```

After running the program, the project folder will look like:

```
StudentManagement
│
├── student_management.py
└── students.db
```

The `students.db` file is created automatically.

Complete Code

Students can also use the following complete version directly after understanding the steps:

```
import tkinter as tk
from tkinter import messagebox
import sqlite3


# ---------------- DATABASE ----------------

connection = sqlite3.connect("students.db")

cursor = connection.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS students (
    student_id INTEGER PRIMARY KEY,
    name TEXT,
    course TEXT,
    email TEXT
)
""")

connection.commit()


# ---------------- FUNCTIONS ----------------

def add_student():

    student_id = id_entry.get()
    name = name_entry.get()
    course = course_entry.get()
    email = email_entry.get()

    if student_id == "" or name == "" or course == "" or email == "":
        messagebox.showwarning(
            "Warning",
            "Please fill all fields."
        )
        return
```

```
    try:

        cursor.execute(
            """
            INSERT INTO students
            (student_id, name, course, email)
            VALUES (?, ?, ?, ?)
            """,
            (student_id, name, course, email)
        )

        connection.commit()

        messagebox.showinfo(
            "Success",
            "Student record added successfully."
        )

        clear_fields()

    except sqlite3.IntegrityError:

        messagebox.showerror(
            "Error",
            "Student ID already exists."
        )


def display_students():

    text_area.delete("1.0", tk.END)

    cursor.execute("SELECT * FROM students")

    records = cursor.fetchall()

    if len(records) == 0:

        text_area.insert(
            tk.END,
            "No student records found."
        )

    else:

        for record in records:

            text_area.insert(
                tk.END,
                "Student ID: " + str(record[0]) +
                "\nName: " + record[1] +
                "\nCourse: " + record[2] +
                "\nEmail: " + record[3] +
                "\n----------------------------\n"
            )


def update_student():

    student_id = id_entry.get()
```

```
    name = name_entry.get()
    course = course_entry.get()
    email = email_entry.get()

    if student_id == "":
        messagebox.showwarning(
            "Warning",
            "Enter Student ID."
        )
        return

    cursor.execute(
        """
        UPDATE students
        SET name=?, course=?, email=?
        WHERE student_id=?
        """,
        (name, course, email, student_id)
    )

    connection.commit()

    if cursor.rowcount > 0:

        messagebox.showinfo(
            "Success",
            "Student record updated successfully."
        )

    else:

        messagebox.showwarning(
            "Warning",
            "Student ID not found."
        )

    clear_fields()


def delete_student():

    student_id = id_entry.get()

    if student_id == "":
        messagebox.showwarning(
            "Warning",
            "Enter Student ID."
        )
        return

    cursor.execute(
        "DELETE FROM students WHERE student_id=?",
        (student_id,)
    )

    connection.commit()

    if cursor.rowcount > 0:

        messagebox.showinfo(
            "Success",
```

```
            "Student record deleted successfully."
        )

    else:

        messagebox.showwarning(
            "Warning",
            "Student ID not found."
        )

    clear_fields()


def clear_fields():

    id_entry.delete(0, tk.END)
    name_entry.delete(0, tk.END)
    course_entry.delete(0, tk.END)
    email_entry.delete(0, tk.END)


# ---------------- GUI ----------------

root = tk.Tk()

root.title("Student Management System")

root.geometry("600x550")

root.resizable(False, False)


# Heading

heading = tk.Label(
    root,
    text="Student Management System",
    font=("Arial", 20, "bold")
)

heading.pack(pady=20)


# Form

form_frame = tk.Frame(root)

form_frame.pack()


tk.Label(
    form_frame,
    text="Student ID:",
    font=("Arial", 12)
).grid(row=0, column=0, padx=10, pady=10)

id_entry = tk.Entry(
    form_frame,
    width=30
)
```

```
id_entry.grid(row=0, column=1)


tk.Label(
    form_frame,
    text="Name:",
    font=("Arial", 12)
).grid(row=1, column=0, padx=10, pady=10)

name_entry = tk.Entry(
    form_frame,
    width=30
)

name_entry.grid(row=1, column=1)


tk.Label(
    form_frame,
    text="Course:",
    font=("Arial", 12)
).grid(row=2, column=0, padx=10, pady=10)

course_entry = tk.Entry(
    form_frame,
    width=30
)

course_entry.grid(row=2, column=1)


tk.Label(
    form_frame,
    text="Email:",
    font=("Arial", 12)
).grid(row=3, column=0, padx=10, pady=10)

email_entry = tk.Entry(
    form_frame,
    width=30
)

email_entry.grid(row=3, column=1)


# Buttons

button_frame = tk.Frame(root)

button_frame.pack(pady=15)


tk.Button(
    button_frame,
    text="Add",
    width=12,
    command=add_student
).grid(row=0, column=0, padx=5)


tk.Button(
```

```
    button_frame,
    text="Display",
    width=12,
    command=display_students
).grid(row=0, column=1, padx=5)


tk.Button(
    button_frame,
    text="Update",
    width=12,
    command=update_student
).grid(row=0, column=2, padx=5)


tk.Button(
    button_frame,
    text="Delete",
    width=12,
    command=delete_student
).grid(row=1, column=0, padx=5, pady=10)


tk.Button(
    button_frame,
    text="Clear",
    width=12,
    command=clear_fields
).grid(row=1, column=1, padx=5, pady=10)


# Display area

text_area = tk.Text(
    root,
    width=65,
    height=12
)

text_area.pack(pady=10)


# Run application

root.mainloop()
```